Sprint 1 Requirements Artifacts

Project 3: WordlePlus System Development — EECS 581: Software Engineering II
Fall 2026 · Professor Hossein Saiedian

Provide detailed artifacts for each Sprint 1 requirement, enabling GTAs to verify completion.

1. Requirement Overviews

IDs	Area	Points
1- 5	Game set up and board	23
6 - 6.4	Input and validation	13
7 - 7.3	Guess evaluation and feedback	12
8 - 8.2	Winning and retry	7
9 - 9.2	Losing and retry	7
10	Game reset	3

2. Details requirement artifacts

Requirement 1 - Display start screen that prompts the user to play

Artifact: Start screen layout: the title “WordlePlus” at the top and the prompt “Press Enter to play” in the center.

src files: main.py ui.py

Expected result: The application opens to start screen with a clear Play option

Requirement 2 - Create window and main game loop

Artifact: Loop outline to initialize pygame, create the program window, read events, update the current screen, and exit the program window

src files: main.py

Expected result: Window stays responsive to any updates/events that occur

Requirement 3 - Load word list from a local TXT file

Artifact: Example word_dictionary lines: APPLE, PLANT, TRANT (one word per line)

src files: word_manager.py word_dictionary.txt
Expected result: A local file is read into a collection of candidate words

Requirement 4 - Select a random target word of given length

Artifact: Sample selection trace: word length = 5 so selected target = APPLE. The selected target word is randomly selected.

src files: word_manager.py

Expected result: A five-letter target word is chosen from the loaded dictionary at the start of each game.

Requirement 5 - Draw game board grid with 5 guess rows

Artifact: Layout of 5 rows by 5 columns of empty tiles, centered on the program window with gaps separating each tiles

src files: ui.py

Expected result: The game screen displays five rows of five letter tiles for the default difficulty.

Requirement 6 - Prompt the user to enter a guess

Artifact: A text prompt on the program window notifying the user to enter a guess

src files: ui.py

Expected result: The playing screen clearly indicates that the user should enter a word. Sample:
"Type a 5-letter word and press Enter"

Requirement 6.1 - Display typed letters in the current row as the user types

Artifact: Type C, A, T and first three tiles show C A T.

src files: input_handler.py game_logic.py ui.py

Expected result: Each typed letter appears in the next available tile.

Requirement 6.2 - Support backspace to delete the last typed letter

Artifact: Before/after APPLE -> APPL after one Backspace.

src files: input_handler.py game_logic.py ui.py

Expected result: Backspace removes the most recent letter in the current guess

Requirement 6.3 - Submit a guess with the Enter key

Artifact: press Enter -> submit current row.
src files: input_handler.py main.py

Expected result: Enter submits the current guess for validation and evaluation.

Requirement 6.4 - Reject incomplete guesses with an error message

Artifact: APP + Enter -> "Enter 5 letters."

src files: game_logic.py ui.py

Expected result: A guess shorter than five letters triggers a visible error and does not consume a row

Requirement 7 - Evaluate a submitted guess against the target word

Artifact: Take the guess and the target word and return results in order of the letters. `Correct` is correct letter and position, `Present` is correct letter but wrong position, or `Absent` is wrong letter.

src files: game_logic.py

Expected result: For a valid guess, the logic compares each letter with the target and produces a five-position result.

Requirement 7.1 - Color tiles green for correct letter in correct position

Artifact: target APPLE, guess PLANE so A at position 3 and E at position 5 are green.

src files: game_logic.py ui.py

Expected result: A letter matching the target at the same index is displayed green.

Requirement 7.2 - Color tiles yellow for correct letter in wrong position
Artifact: Sample: target APPLE, guess PLANE so P and L are yellow.

src files: game_logic.py ui.py

Expected result: Misplaced letters display yellow when an unmatched occurrence exists.

Requirement 7.3 - Color tiles grey for letters not in the word

Artifact: Sample: target APPLE, guess PLANE so N is gray.

src files: game_logic.py ui.py

Expected result: Letters absent from the target, or extra unmatched duplicates, show grey.

Requirement 8 - Detect a win when the guess matches the target word

Artifact: Input/output example: target APPLE, guess APPLE -> evaluation [green, green, green, green, green] which then leads to a winning game state.

src files: game_logic.py

Expected result: The game detects a win on exact match with the target and moves on to the win screen. There is no additional input.

Requirement 8.1 - Display win screen after a correct guess

Artifact: A screenshot of the win screen with the retry button.

src files: end_screen.py main.py

Expected result: The win screen appears right after the winning screen is evaluated and the row is colored.

Requirement 8.2 - Offer retry option on the win screen
Artifact: A screenshot of the win screen with the retry button.

src files: end_screen.py main.py

Expected result: The win screen shows a retry option that starts a new game when clicked.

Requirement 9 - Detect a loss when all guesses are used
Artifact: Sample: target APPLE, five valid guesses do not match: PLANT, CRANE, GRAPE, MAPLE, AMPLE. Then after the fifth guess, the game state is a loss. Before the fifth guess, the game state is "playing".

src files: game_logic.py

Expected result: The game detects a loss when the last row is used without a match. There is no additional input.

Requirement 9.1 - Display lose screen that reveals the target word
Artifact: A screenshot of the losing screen after a trace. The trace can be: Sample: target APPLE, five valid guesses do not match: PLANT, CRANE, GRAPE, MAPLE, AMPLE. Then it must show that the target was APPLE.

src files: end_screen.py main.py

Expected result: The loss screen appears after the final incorrect guess and then displays the correct target word.

Requirement 9.2 - Offer retry option on the lose screen
Artifact: A screenshot of the loss screen with the retry button.

src files: end_screen.py main.py

Expected result: The loss screen shows a retry option that starts a new game when clicked.

Requirement 10 - Reset game state and pick a new word on retry
Artifact: Before or after a trace for a retry click. This follows the state of the game and the board before and after the retry click.

State	Before Retry(after a loss)	After Retry
Board	5 rows filled with colored tiles	All tiles empty
Guess count/current row	5/row 5	0/row 1
Current typed letters	Cleared	Empty
Target word	Apple	A new word
Game state	Lose	playing

Could include a screenshot of the game in “playing” state after clicking retry to show an empty board.

src files: game_logic.py word_manager.py main.py

Expected result: The retry option shows a clean board, resets the guess count, and selects a new random target word. Since the choice is random, the new word may match previous words. Must verify the new word over retries.